

组成原理课程第五次实报告

实验名称：单-多周期 CPU

学号：2312331 姓名：王一诺 班次：张金老师

1、实验内容说明

请根据实验指导手册中的单周期 CPU 和多周期 CPU 实验内容,完完成如下任务并撰写实验报告:

- 1、针对单周期 CPU 实验,复现并验证功能,同时对三种类型的 MIPS 指令,挑 1~2 条具体分析总结其执行过程。
- 2、针对多周期 CPU 实验,请认真分析指令 ROM 中的指令执行情况,找到存在的 bug 并修复,实验报告中总结寻找 bug 和修 复 bug 的过程。
- 3、将 ALU 实验中扩展的三种运算,以自定义 MIPS 指令的开式,添加到多周期 CPU 实验代码中,并自行编写指令存储到指 令 ROM,然后验证正确性,波形验证或实验箱上箱验证均可。

注意:单周期 CPU 使用异步存储器(.v)文件格式,多周期 CPU 使用的是同步存储器(IP 核形式),二者不要弄混。多周期实验中,每一个 IP 核请自行创建到项目中,不要使用源码中的 docp 文件。

2、单周期

2.1 实验目的

1. 理解 MIPS 指令结构,理解 MIPS 指令集中常用指令的功能和编码,学会对这些指令进行归纳分类。
2. 了解熟悉 MIPS 体系的处理器结构,如延迟槽,哈佛结构的概念。
3. 熟悉并掌握单周期 CPU 的原理和设计。
4. 进一步加强运用 verilog 语言进行电路设计的能力。
5. 为后续设计多周期 cpu 的实验打下基础。

2.2 实验原理图

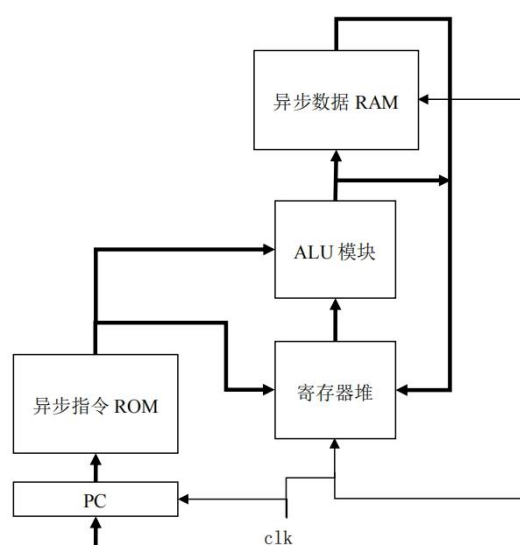


图 7.1 单周期 CPU 的大致框图

单周期 CPU 是指一条指令的所有操作在一个时钟周期内执行完。设计中所有寄存

器和存储器都是异步读同步写的，即读出数据不需要时钟控制，但写入数据需时钟控制。

故单周期 CPU 的运作即：在一个时钟周期内，根据 PC 值从指令 ROM 中读出相应的指令，将指令译码后从寄存器堆中读出需要的操作数，送往 ALU 模块，ALU 模块运算得到结果。

2.3 实验步骤

```
//-----{访存}begin-----
wire [3:0] dm_wen;
wire [31:0] dm_addr;
wire [31:0] dm_wdata;
wire [31:0] dm_rdata;

assign dm_wen = {4(inst_SW)} & resetn; // 内存写使能, 非resetn状态下有效
assign dm_addr = alu_result;           // 内存写地址, 为ALU结果
assign dm_wdata = rt_value;           // 内存写数据, 为rt寄存器值
data_ram data_ram_module(
    .clk (clk), // I, 1, 时钟
    .wen (dm_wen), // I, 1, 写使能
    .addr (dm_addr[6:2]), // I, 32, 读地址
    .wdata (dm_wdata), // I, 32, 写数据
    .rdata (dm_rdata), // O, 32, 读数据

    //display mem
    .test_addr(mem_addr[6:2]),
    .test_data(mem_data)
);
//-----{访存}end-----
```

Name	Value
clk	1
resetn	1
mem_addr[31:0]	00000000000000000000000000000000
mem_data[31:0]	XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
cpu_pc[31:0]	00000000000000000000000000000000
cpu_inst[31:0]	00000000000000000000000000000000

原代码存在写使能处理有误的 bug，可以看到错误地将四位信号与一位信号相与，这导致只能向内存写入最低位，将其修改为下述代码，可以成功运行。

```
//-----{访存}begin-----//
wire [3:0] dm_wen;
wire [31:0] dm_addr;
wire [31:0] dm_wdata;
wire [31:0] dm_rdata;

assign dm_wen = {4(inst_SW)} & {4(resetn)}; // 内存写使能, 非resetn状态下有效
assign dm_addr = alu_result; // 内存写地址, 为ALU结果
assign dm_wdata = rt_value; // 内存写数据, 为rt寄存器值
data_ram data_ram_module(
    .clk (clk), // I, 1, 时钟
    .wen (dm_wen), // I, 1, 写使能
    .addr (dm_addr[6:2]), // I, 32, 读地址
    .wdata (dm_wdata), // I, 32, 写数据
    .rdata (dm_rdata), // O, 32, 读数据

    //display mem
    .test_addr(mem_addr[6:2]),
    .test_data(mem_data)
);
//-----{访存}end-----//
```

原代码设置了 20 条指令的执行，我们上箱复现，这里只截取几段展示：

PC: 00000018	INST: 00A23027
MADDR: 00000014	MDATA: 0000000D
REG00: 00000000	REG01: 00000001
REG02: 00000010	REG03: 00000011
REG04: 00000004	REG05: 0000000D
REG06: 00000000	REG07: 00000000
REG08: 00000000	REG09: 00000000
REG0A: 00000000	REG0B: 00000000

sw \$5, #19(\$1)指令结束后如上图所示，至此完成了对寄存器 \$1~\$5 的赋值，并将\$5寄存器的值存在了内存 Mem[0000_0014H]中；

LOONGSON	
PC: 0000002C	INST: 11210002
MADDR: 0000001C	MDATA: 00000011
REG00: 00000000	REG01: 00000001
REG02: 00000010	REG03: 00000011
REG04: 00000004	REG05: 0000000D
REG06: FFFFFFFE2	REG07: FFFFFFFF3
REG08: 00000011	REG09: 00000001
REG0A: 00000000	REG0B: 00000000
REG0C: 00000000	REG0D: 00000000

sw \$8, #28(\$0) 指令结束后如上图所示，到此完成了对寄存器 \$6~\$8 的赋值，并将 \$8 寄存器的值存在了内存 Mem[0000_001CH]中。此后的不再赘述。

2.4 三种类型的 MIPS 指令

MIPS 指令集主要分为三种类型：R 型（寄存器型）、I 型（立即数型）和 J 型（跳转型）。

1>R 型指令

格式：opcode (6) | rs (5) | rt (5) | rd (5) | shamt (5) | funct (6)

其中，op 段 (6b)：恒为 0b000000；rs (5b)、rt (5b)：两个源操作数所在的寄存器号；rd (5b)：目的操作数所在的寄存器号；shamt (5 位)：位移量，决定移位指令的移位位数；func (6b)：功能位，决定 R 型指令的具体功能。

特点：所有操作数均为寄存器，用于算术/逻辑运算（如加减、移位）。

示例：00411821。该操作码对应的汇编指令为 addu \$3, \$2, \$1。我们将该操作码展成二进制：0000_0000_0100_0001_1000_0010_0001，前 6 位 000000 代表 R 型指令，5 位 00010 对应 \$2 寄存器，接着 00001 对应 \$1 寄存器，接着为 00011 对应 \$3 寄存器，00000 表示不做移位，100001 是功能码对应的就是 addu 指令。也就是说，此处执行的操作为：将 \$2 寄存器与 \$1 寄存器相加的值保存在 \$3 寄存器中。

从这个例子可以概括执行过程：从指令存储器中取指令，更新 PC；ALU 根据 funct 字段确定 ALU 的功能；从寄存器堆中读出寄存器 rs 和 rt，对它们进行确定的 ALU 操作；最后将运算结果写入到 rd 字段对应的目标寄存器。

2>I 型指令

格式：opcode (6) | rs (5) | rt (5) | immediate (16)

其中，op 段 (6b)：决定 I 型指令类型；rs (5b)：是第一个源操作数所在的寄存器号；rt (5b)：是第二个源操作数所在的寄存器号或目的操作数所在的寄存器编号。immediate (16b)：立即数或地址。

特点：包含 16 位立即数，用于加载/存储、条件分支等。

示例：24010001。该操作码对应的汇编指令为 addiu \$1, \$0, #1。将操作码转成二进制：0010_0100_0000_0001_0000_0000_0000_0001，前 6 位 001001 表明进行 addui 立即数加法运算，接下来 5 位 00000 对应 \$0 寄存器，接着 00001 对应 \$1 寄存器，最后 16 位表明立即数是 1。也就是说，此处执行的操作为：把 \$0 寄存器的值与立即数 1 进行相加，结果存在 \$1 寄存器。

从这个例子可以概括执行过程：取指：从指令存储器中读取指令，解析字段；译码：读取基址寄存器 rs 的值；执行：计算内存地址；访存：根据计算出的地址，从数据存储器中读取数据；最后写回，将读取的数据写入目标寄存器 rt。

3>J 型指令

格式：opcode (6) | address (26)

特点：26 位目标地址，用于无条件跳转。

示例：08000000。对应汇编指令为 j 00H。转成二进制为

0000_1000_0000_0000_0000_0000_0000_0000，前 6 位 000010 对应指令 j，后 26 位表示跳转到 0 号处。

可以总结出执行过程：从指令存储器中取指令，更新 PC，然后取出 address 字段作为目标跳转地址，将目标跳转地址存入 PC 中。

3、多周期

3.1 实验目的

1. 在单周期 CPU 实验完成的提前下，理解多周期的概念。
2. 熟悉并掌握多周期 CPU 的原理和设计。
3. 进一步提升运用 verilog 语言进行电路设计的能力。
4. 为后续实现流水线 cpu 的课程设计打下基础。

3.2 实验原理图

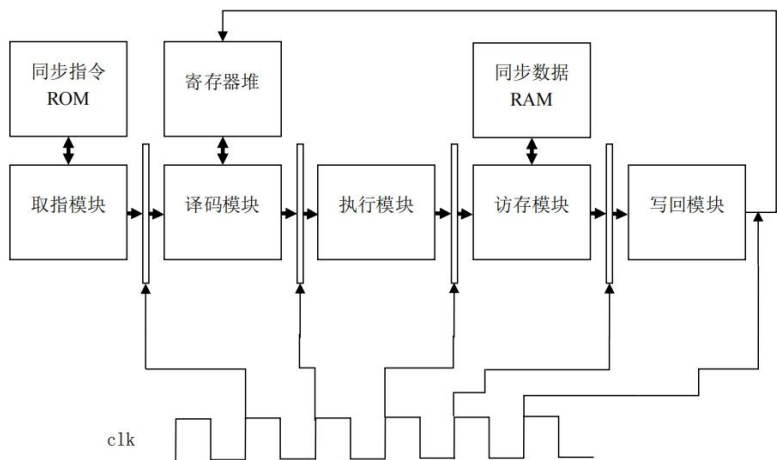


图 8.1 多周期 CPU 的大致框图

多周期 CPU 是指，一条指令需要花费多个周期才能完成所有操作，在每个周期内只做一部分操作，比如：取指、译码、执行、访存、写回，此时，一条指令执行完，共需 5 个周期，每个周期只做一部分操作。这样，每个时钟周期内 CPU 需要做的工作就变少，因此频率可以更高，且每个部件做的事情单一了，因此 CPU 可以运行的更快。

3.3 实验步骤

add.v	2016/4/29 12:58	V 文件	1 KB
alu.v	2025/5/13 19:48	V 文件	10 KB
decode.v	2025/5/13 20:08	V 文件	13 KB
exe.v	2025/5/13 20:39	V 文件	3 KB
fetch.v	2016/5/6 16:44	V 文件	3 KB
inst_rom.mif	2025/5/13 21:20	MIF 文件	2 KB
mem.v	2016/5/6 17:25	V 文件	5 KB
multi_cycle_cpu.v	2016/5/9 21:08	V 文件	11 KB
multi_cycle_cpu.xdc	2017/2/9 11:52	XDC 文件	4 KB
multi_cycle_cpu_display.v	2016/11/17 23:20	V 文件	7 KB
regfile.v	2016/4/30 11:02	V 文件	5 KB
tb.v	2016/4/30 11:07	V 文件	2 KB
wb.v	2016/5/6 17:28	V 文件	2 KB

导入如上源代码，rom 与 ram 需要手动创建 IP 核文件并进行配置。不过直接照搬之前的 IP 核配置是不可取的，存在的周期不匹配问题（输出寄存器与数据输入的时钟不同步）导致仿真出现 X，改用如下 IP 核配置即可：

data_ram:

Customize IP

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

Show disabled ports

Component Name data_ram

Basic Port A Options Port B Options Other Options Summary

Interface Type Native Generate address interface with 32 bits

Memory Type True Dual Port RAM Common Clock

ECC Options

ECC Type No ECC

Error Injection Pins Single Bit Error Injection

Write Enable

Byte Write Enable

Byte Size (bits) 8

Algorithm Options

Defines the algorithm used to concatenate the block RAM primitives. Refer datasheet for more information.

Algorithm Minimum Area

Primitive 8kx2

OK Cancel

Customize IP

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

Show disabled ports

Component Name data_ram

Basic Port A Options Port B Options Other Options Summary

Memory Size

Write Width 32 Range: 8 to 4096 (bits)

Read Width 32

Write Depth 256 Range: 2 to 1048576

Read Depth 256

Operating Mode Write First Enable Port Type Always Enabled

Port A Optional Output Registers

Primitives Output Register Core Output Register

SoftECC Input Register REGCEA Pin

Port A Output Reset Options

RSTA Pin (set/reset pin) Output Reset Value (Hex) 0

Reset Memory Latch Reset Priority CE (Latch or Register Enable)

READ Address Change A

Read Address Change A

OK Cancel

Inst_rom:

Customize IP

Block Memory Generator (8.4)

Documentation
IP Location
Switch to Defaults

IP Symbol

Power Estimation

☒ Show disabled ports

+

AR_SLAVE_S_AXI

+

ARLite_SLAVE_S_AXI

+

BRAM_PORTA

+

BRAM_PORTB

regres

regdata

injectchldren

injectchldren

scppace

sleep

shutdown

s_nack

s_nresets

s_axi_injectchldren

s_axi_injectchldren

other

dataen

readback[3:0]

rdst_busy

rdst_busy

s_axi_writer

s_axi_writer

s_axi_readback[3:0]

Component Name

inst_rom

Basic

Port A Options

Other Options

Summary

Interface Type

Native

☐ Generate address interface with 32 bits

Memory Type

Single Port ROM

☐ Common Clock

ECC Options

ECC Type

No ECC

☐ Error Injection Pins

Single Bit Error Injection

Write Enable

☐ Byte Write Enable

Byte Size (bits)

9

Algorithm Options

Defines the algorithm used to concatenate the block RAM primitives.
Refer datasheet for more information.

Algorithm

Minimum Area

Primitive

8kx2

OK

Cancel

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☒ Show disabled ports



Component Name inst_rom

Basic Port A Options Other Options Summary

Memory Size

Port A Width 32 Range: 1 to 4608 (bits)
 Port A Depth 256 Range: 2 to 1048576
 The Width and Depth values are used for Read Operation in Port A

Operating Mode Write First Enable Port Type Always Enabled

Port A Optional Output Registers

☐ Primitives Output Register ☐ Core Output Register
☐ SoftECC Input Register ☐ REGCEA Pin

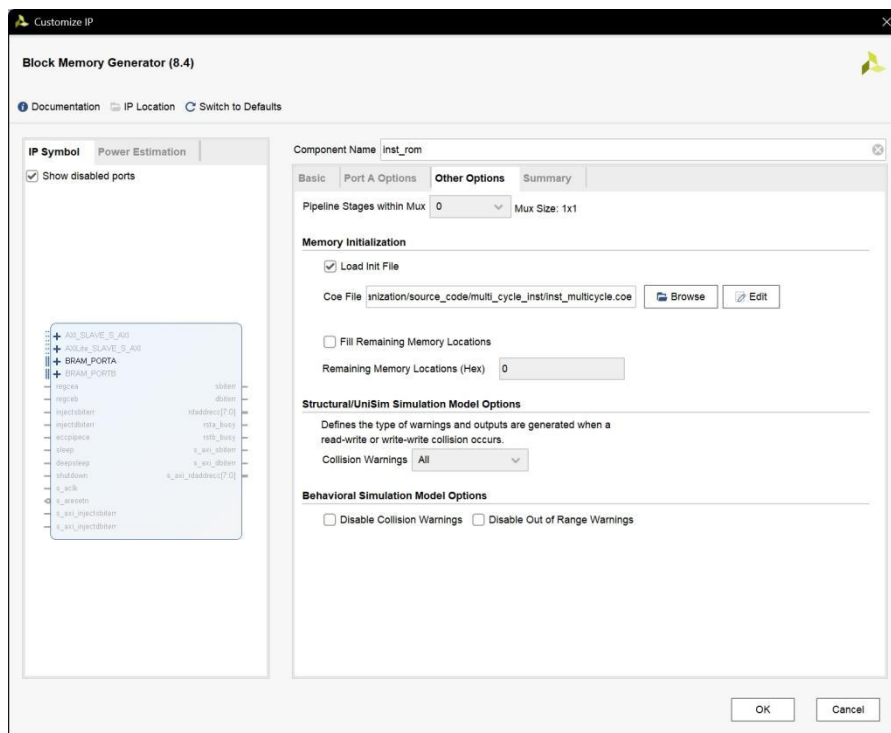
Port A Output Reset Options

☐ RSTA Pin (set/reset pin) Output Reset Value (Hex) 0
☐ Reset Memory Latch Reset Priority CE (Latch or Register Enable)

READ Address Change A

☐ Read Address Change A

OK Cancel



注意这里还导入了写有指令程序的. coe 文件，后面我们新增运算的指令也要加到这个文件里并重新导入。

至此完成了多周期 CPU 的正常实现，接下来我们尝试把上次实验在 ALU 里新增的三种运算也加到这个 CPU 里，并测试验证。

首先是对 alu.v 的修改，如下所示。加入新的三种运算，要注意控制信号也要随之改变，这里直接改用四位二进制信号对增添后的运算进行控制选择：

```
module alu(
    input  [3:0] alu_control, // ALU 控制信号
    input  [31:0] alu_src1,   // ALU 操作数 1,为补码
    input  [31:0] alu_src2,   // ALU 操作数 2, 为补码
    output [31:0] alu_result  // ALU 结果
);
.....
// ALU 控制信号
.....
// 新增三种运算
wire alu_sgt; //有符号比较，大于置位，复用加法器做减法
wire alu_nxor; //按位同或
wire alu_nand; //按位与非
.....
assign alu_add = ( alu_control == 4'b0001);
assign alu_sub = (alu_control == 4'b0010 ) ;
assign alu_slt = ( alu_control == 4'b0011);
assign alu_sltu = (alu_control == 4'b0100 ) ;
assign alu_and = ( alu_control == 4'b0101);
assign alu_nor = (alu_control == 4'b0110 ) ;
```

```

assign alu_or = ( alu_control == 4'b0111);
assign alu_xor = (alu_control == 4'b1000 ) ;
assign alu_sll = ( alu_control == 4'b1001);
assign alu_srl = (alu_control == 4'b1010 ) ;
assign alu_sra = ( alu_control == 4'b1011);
assign alu_lui = (alu_control == 4'b1100 ) ;
assign alu_sgt = ( alu_control == 4'b1101);
assign alu_nxor = (alu_control == 4'b1110 ) ;
assign alu_nand = ( alu_control == 4'b1111);

.....
// 新增三种运算的结果
wire [31:0] sgt_result;
wire [31:0] nxor_result;
wire [31:0] nand_result;

.....
assign nxor_result = ~xor_result;           // 同或结果为异或取反
assign nand_result = ~and_result;          // 与非结果为与结果按位取反
// sgt 结果
// 不满足小于置位且两操作数不相等的就是大于置位
wire adder_zero;
assign adder_zero = |adder_result;
assign sgt_result[31:1] = 31'd0;
assign sgt_result[0] = ~((alu_src1[31] & ~alu_src2[31]) | (~(alu_src1[31]^alu_src2[31]) &
adder_result[31])) & adder_zero;

.....
// 选择相应结果输出
assign alu_result = (alu_add|alu_sub) ? add_sub_result[31:0] :
    alu_slt           ? slt_result :
    alu_sltu          ? sltu_result :
    alu_and            ? and_result :
    alu_nor            ? nor_result :
    alu_or             ? or_result  :
    alu_xor            ? xor_result :
    alu_sll            ? sll_result :
    alu_srl            ? srl_result :
    alu_sra            ? sra_result :
    alu_lui            ? lui_result :
    alu_sgt            ? sgt_result :// 有符号数,大于置位比较
    alu_nxor           ? nxor_result :// 按位同或
    alu_nand           ? nand_result :// 按位与非
    32'd0;

```

对 ALU 的修改为上个实验的内容，此处不再赘述。因为 ALU 的变动，其顶层模块 exe.v 也需要小小的修改，这里只需要把 alu 控制信号改为 4 位位宽即可：


```
wire [3:0] alu_control;
```

接下来还要对 decode 模块进行修改，为我们新增的三种运算设置对应的指令并加入到译码逻辑中。先声明我们要增添的三个指令：

```
wire inst_SGT, inst_NXOR, inst_NAND;
```

然后，为我们的新增运算设置指令码。我新增的大于置位比较 sgt、按位同或 nxor 和按位与非 nand 均为 R 型指令，按照 R 型指令的结构进行定义。需要注意，最后 6 位功能码要与已经出现的功能码不同。如下所示：

```
assign inst_SGT = op_zero & sa_zero & ( funct == 6'b001001); //大于置位
assign inst_NXOR = op_zero & sa_zero & (funct == 6'b001010); //按位同或
assign inst_NAND = op_zero & sa_zero & (funct == 6'b001011); //按位与非
```

然后是对 alu 控制信号、指令分类和选取的更改，如下所示：

```
wire inst_sgt, inst_nxor, inst_nand;
assign inst_sgt = inst_SGT;
assign inst_nxor = inst_NXOR;
assign inst_nand = inst_NAND;
.....
//依据目的寄存器号分类
wire inst_wdest_rt; // 寄存器堆写入地址为 rt 的指令
wire inst_wdest_31; // 寄存器堆写入地址为 31 的指令
wire inst_wdest_rd; // 寄存器堆写入地址为 rd 的指令
assign inst_wdest_rt = inst_imm_zero | inst_ADDIU | inst_SLTI | inst_SLTIU | inst_load;
assign inst_wdest_31 = inst_JAL;
assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SLT | inst_SLTU | inst_JALR |
inst_AND | inst_NOR | inst_OR | inst_XOR | inst_SLL | inst_SLLV | inst_SRA | inst_SRAV |
inst_SRL | inst_SRLV | inst_SGT | inst_NXOR | inst_NAND;
.....
assign alu_control = ( inst_add ? 4'b0001:
                    inst_sub ? 4'b0010 :
                    inst_slt ? 4'b0011:
                    inst_sltu ? 4'b0100 :
                    inst_and ? 4'b0101:
                    inst_nor ? 4'b0110 :
                    inst_or ? 4'b0111:
                    inst_xor ? 4'b1000 :
                    inst_sll ? 4'b1001:
                    inst_srl ? 4'b1010 :
                    inst_sra ? 4'b1011:
                    inst_lui ? 4'b1100 :
                    inst_sgt ? 4'b1101:
                    inst_nxor ? 4'b1110 :
                    inst_nand ? 4'b1111:
                    4'b0000 );
```

至此我们完成了多周期 CPU 的复现和三种 alu 运算的新增,为了能够测试新增三种运算是否正确执行,还需要把三个运算对应的指令及其机器码添加到.coe (十六进制机器码)和.mif (二进制指令)文件中,这里我添加到了结尾跳转指令之前。

SGT 指令为 000000 00010 00001 00011 00000 001001, R 型指令前六位全 0, 接着五位 00010 表示\$2 寄存器为第一个操作数, 接着五位 00001 表示\$1 寄存器为第二个操作数, 接着五位 00011 表示\$3 寄存器存放着运算结果, 然后五位 00000 表示特殊操作最后六位是标识该运算的功能码。对应十六进制机器码为 00411809, 加入.coe 中。

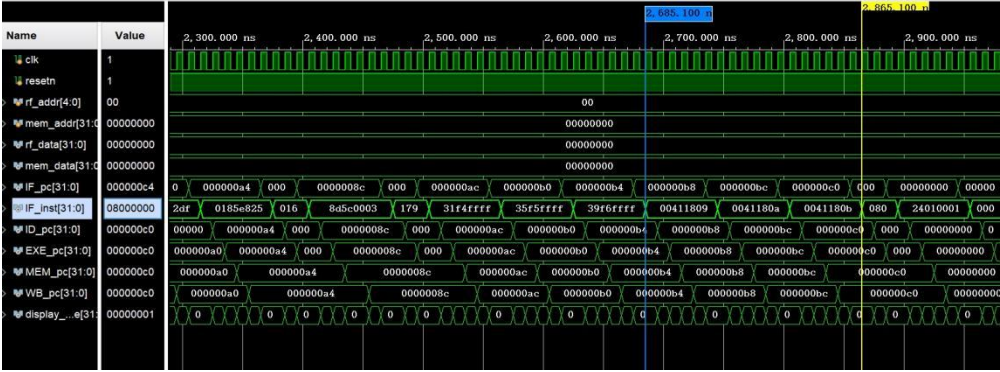
nxor 指令为 000000 00010 00001 00011 00000 001010, 与 SGT 类似, 不再赘述。对应十六进制机器码为 0041180a, 加入.coe 中。

nand 指令为 000000 00010 00001 00011 00000 001011, 与 SGT 类似, 不再赘述。对应十六进制机器码为 0041180b, 加入.coe 中。

3.3 实验结果

3.3.1 仿真

对修改后的项目进行仿真,可见设计的指令正确执行,新增的三个运算也成功加入,如下图所示:



3.3.2 上箱

生成二进制流文件后,上箱烧制程序,按下按键即可单步执行。多周期实验设置了更多的指令,运行效果与单周期类似,这里不再展示,只观察新增的指令是否正确执行:

LUONG SON	
IF PC: 000000BC	IF IN: 00411809
ID PC: 000000B8	EXEPC: 000000B8
MEMPC: 000000B8	WB PC: 000000B8
MADDR: 00000000	MDATA: 00000000
STATE: 00000001	
REG00: 00000000	REG01: 00000008
REG02: 00000010	REG03: 00000001
REG04: 00000004	REG05: 0000000D
REG06: FFFFFFFE2	REG07: FFFFFFFF3
REG08: 00000011	REG09: 00000000
REG0A: 00000011	REG0B: 0000008C
REG0C: 000C0000	REG0D: 00000000
REG0E: FFFFFFFE20	REG0F: FFFFFFFF88
REG10: FFFFFFFF	REG11: 00000000
REG12: FFFFFFFF88	REG13: 00000088
REG14: 0000FF88	REG15: FFFFFFFF
REG16: FFFF0077	REG17: 00000000
REG18: 00000001	REG19: 00000001
REG1A: 0000000C	REG1B: 00000018
REG1C: 000C880D	REG1D: 000C000D
REG1E: 00000000	REG1F: 00000054

左图所示为 00411809 指令执行之后的结果。该 SGT 指令是将 \$2 寄存器与 \$1 寄存器的比较结果存入 \$3 寄存器中。

由图可见 \$2 寄存器为 00000010, \$1 寄存器为 00000008, \$2 寄存器所存比 \$1 寄存器的大,故 \$3 寄存器置 1。

验证正确。

L0ONGSON	
IF PC:000000C0	IF IN:0041180B
ID PC:000000BC	EXEPC:000000BC
MEMPC:000000BC	WB PC:000000BC
MADDR:00000000	MDATA:00000000
STATE:00000001	
REG00:00000000	REG01:00000008
REG02:00000010	REG03:FFFFFFE7
REG04:00000004	REG05:0000000D
REG06:FFFFFFE2	REG07:FFFFFFF3
REG08:00000011	REG09:00000000
REG0A:00000011	REG0B:0000008C
REG0C:000C0000	REG0D:00000000
REG0E:FFFFFFE20	REG0F:FFFFFFF88
REG10:FFFFFFF88	REG11:00000000
REG12:FFFFFFF88	REG13:00000088
REG14:0000FF88	REG15:FFFFFFF88
REG16:FFFFF0077	REG17:00000000
REG18:00000001	REG19:00000001
REG1A:0000000C	REG1B:00000018
REG1C:000C880D	REG1D:000C000D
REG1E:00000000	REG1F:00000054

左图所示为 0041180a 指令执行之后的结果。该 nxor 指令是将 \$2 寄存器与 \$1 寄存器的按位同或结果存入 \$3 寄存器中。

由图可见 \$2 寄存器为 00000010, \$1 寄存器为 00000008, 所以该运算为
0000 0000 0000 0000 0000 0000 0001 0000
与

0000 0000 0000 0000 0000 0000 0000 1000
按位同或, 结果为

1111 1111 1111 1111 1111 1111 1110 0111

所以 \$3 寄存器应为 FFFFFFFE7。

验证正确。

L0ONGSON	
IF PC:000000C4	IF IN:08000000
ID PC:000000C0	EXEPC:000000C0
MEMPC:000000C0	WB PC:000000C0
MADDR:00000000	MDATA:00000000
STATE:00000001	
REG00:00000000	REG01:00000008
REG02:00000010	REG03:FFFFFFF7
REG04:00000004	REG05:0000000D
REG06:FFFFFFE2	REG07:FFFFFFF3
REG08:00000011	REG09:00000000
REG0A:00000011	REG0B:0000008C
REG0C:000C0000	REG0D:00000000
REG0E:FFFFFFE20	REG0F:FFFFFFF88
REG10:FFFFFFF88	REG11:00000000
REG12:FFFFFFF88	REG13:00000088
REG14:0000FF88	REG15:FFFFFFF88
REG16:FFFFF0077	REG17:00000000
REG18:00000001	REG19:00000001
REG1A:0000000C	REG1B:00000018
REG1C:000C880D	REG1D:000C000D
REG1E:00000000	REG1F:00000054

左图所示为 0041180b 指令执行之后的结果。该 nand 指令是将 \$2 寄存器与 \$1 寄存器的按位与非结果存入 \$3 寄存器中。

由图可见 \$2 寄存器为 00000010, \$1 寄存器为 00000008, 所以该运算为
0000 0000 0000 0000 0000 0000 0001 0000
与

0000 0000 0000 0000 0000 0000 0000 1000
按位与非, 结果为

1111 1111 1111 1111 1111 1111 1111 1111

所以 \$3 寄存器应为 FFFFFFFF。

验证正确。

4、总结感想

1. 通过本次实验, 对 CPU 的设计和结构有了直观具体的感受和更深的认识, 对于单周期和多周期 CPU 的原理和实现收获了一定的理解。

2. 对于设计与调试过程中 bug 的发现、分析和改动有了更深的认识, 同时也发现了这方面是我的薄弱项, 以后自己进行设计时要注意, 一方面要有明确的设计思路与心细的代码实现以避免 bug 出现, 另一方面可以多多留心调试与改 bug 过程以增强自己的综合能力。

3. 熟悉了 IP 核的创建与配置, 对异步和同步存储器有了更深刻的认识。

4. 对指令的设计和其所代表的意义有了认知和理解, 对于指令的执行过程也有了一定认识。